Lab(Option B: JavaScript): CRUD Operations

Skills Network

Estimated Time Needed: 60 mins

In this lab, you will learn how to create a **Product's list** using an Express.js server. Your application should allow you to add a product, retrieve the products, retrieve a specific product with its id, update a specific product with its id, and delete a product with its id. All these operations will be achieved through the REST API endpoints in your Express.js server.

You will create an application with API endpoints to perform Create, Retrieve, Update, and Delete operations on the above data using Express.js.

You will use CURL and POSTMAN to test the implemented endpoints.

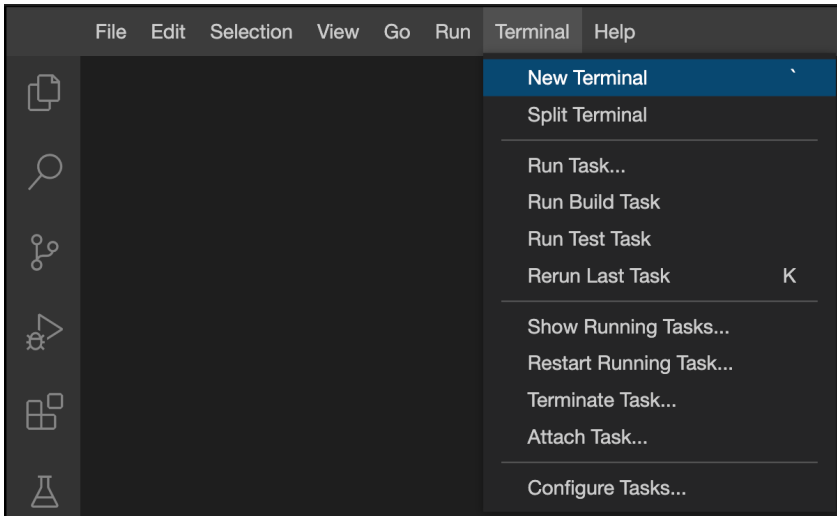
Objectives:

After completing this lab you will be able to:

- Create API endpoints to perform Create, Retrieve, Update, and Delete operations on transient data with an Express.js server.
- Create REST API endpoints, and use POSTMAN to test your REST APIs.

Set-up : Create application

1. Open a terminal window by using the menu in the editor: **Terminal** > **New Terminal**.



2. Change to your project folder, if you are not in the project folder already.

```
cd /home/project
```

3. Run the following command to clone the Git repository that contains the starter code needed for this lab, if it doesn't already exist.

```
[ ! -d 'ihsgq-crud_microservices_js' ] && git clone https://github.com/ibm-developer-skills-network/ihsgq-crud_microservices_js.git
```

4. Change to the directory **ihsgq-crud_microservices_js/CRUD** to start working on the lab.

```
cd ihsgq-crud_microservices_js/CRUD
```

5. List the contents of this directory to see the artifacts for this lab.

```
ls
```

6. Run the following command on the terminal to install the packages that are required.

```
npm install express
```

Exercise 1: Understand the server application

1. In the Files Explorer open the **ihsgq-crud_microservices_js/CRUD** folder and view **products.js**.
2. You first need to import the packages required to create REST APIs with Express.

```
const express = require('express');
```

3. You can then create the Express application, which will service all the REST APIs for adding, retrieving, updating, and deleting products.

```
const app = express();
const port = 5000;
// Middleware
app.use(express.json());
```

4. The code has precreated products added to the list. These are defined in the following code.

```
let products = [
    { id: 143, name: 'Notebook', price: 5.49 },
    { id: 144, name: 'Black Marker', price: 1.99 }
];
```

5. Now that the server is defined, you will create the REST API endpoints and define the routes or paths, one for each of the following operations.

- Retrieve all the products - `GET` Request Method
- Retrieve a product by its id - `GET` Request Method
- Add a product - `POST` Request Method
- Update a product by its id - `PUT` Request Method
- Delete a product by its id - `DELETE` Request Method

Add the following code to products.js in the space provided.

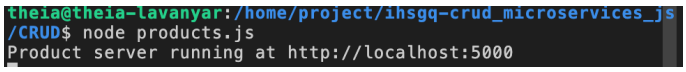
```
// Example request - http://localhost:5000/products
app.get('/products', (req, res) => {
    res.json(products);
});
// Example request - http://localhost:5000/products/144 - with method GET
app.get('/products/:id', (req, res) => {
    const id = parseInt(req.params.id);
    const product = products.find(x => x.id === id);
    if (product) {
        res.json(product);
    } else {
        res.status(404).json({ message: 'Product not found' });
    }
});
// Example request - http://localhost:5000/products - with method POST
app.post('/products', (req, res) => {
    const newProduct = req.body;
    products.push(newProduct);
    res.status(201).send(); // Return a 201 status with no content
});
// Example request - http://localhost:5000/products/144 - with method PUT
app.put('/products/:id', (req, res) => {
    const id = parseInt(req.params.id);
    const updatedProduct = req.body;
    const product = products.find(x => x.id === id);
    if (product) {
        Object.assign(product, updatedProduct); // Update product properties
        res.status(204).send(); // Return a 204 status with no content
    } else {
        res.status(404).json({ message: 'Product not found' });
    }
});
// Example request - http://localhost:5000/products/144 - with method DELETE
app.delete('/products/:id', (req, res) => {
    const id = parseInt(req.params.id);
    const productIndex = products.findIndex(x => x.id === id);
    if (productIndex !== -1) {
        products.splice(productIndex, 1); // Remove product from array
        res.status(204).send(); // Return a 204 status with no content
    } else {
        res.status(404).json({ message: 'Product not found' });
    }
});
```

Run and test the server with CURL

1. In the terminal, run the Node.js server using the following command.

```
node products.js
```

The server starts running and listening at port 5000.



2. Open another `Terminal` by clicking the Terminal menu and selecting **New Terminal**.

3. In the new terminal, run the following command to access the **http://localhost:5000/products** API endpoint. `curl` command stands for `Client URL` and is used as command line interfacing with the server serving REST API endpoints. It is, by default, a `GET` request.

```
curl http://localhost:5000/products
```

This returns a JSON with the products that have been preloaded.

4. In the terminal, run the following command to add a product to the list. This will be a POST request to which you will pass the product parameter as a JSON.

```
curl -X POST -H "Content-Type: application/json" \
    -d '{"id": 145, "name": "Pen", "price": 2.5}' \
    http://localhost:5000/products
```

This command will not return any output. It will add the product to the list of products.

5. Verify if the product is added by running the following command.

```
curl http://localhost:5000/products/145
```

This should return the details of the product you just added with a POST request.

Using POSTMAN to test the REST API endpoints

While the `GET` endpoints are easy to test with `curl` using a command line interface, the `POST`, `PUT` and `DELETE` commands can be cumbersome. To circumvent this problem, you can use Postman, which is a software that is available as a service.

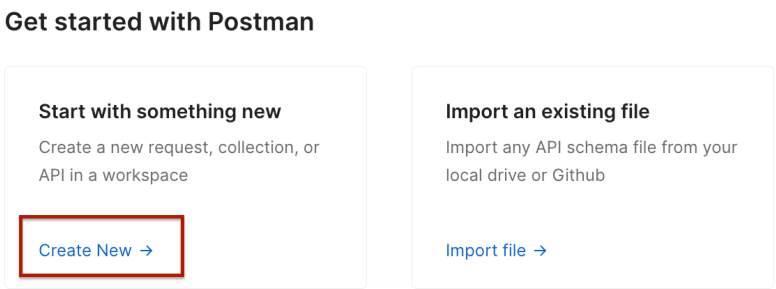
Postman will need remote access to the server as it is an external entity. To get the URL click the following button.

Launch Application

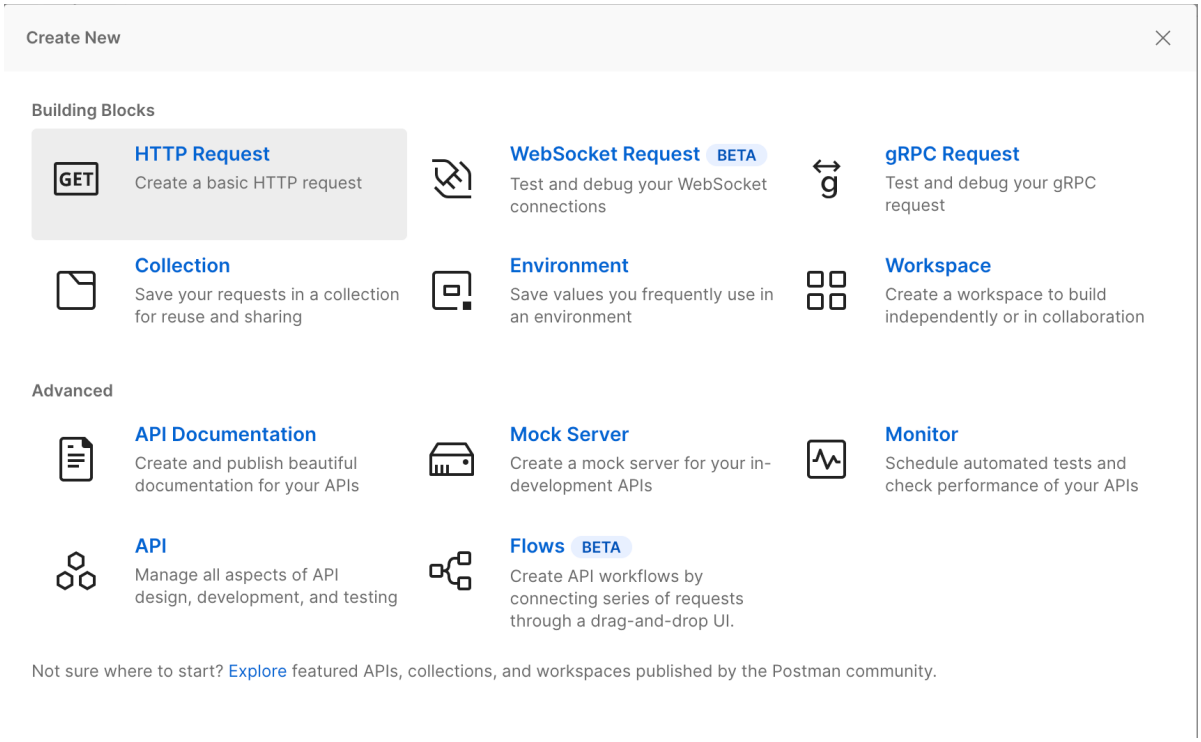
Copy the URL into a notepad or other text editors. Go to https://www.postman.com/ and sign up before you start using POSTMAN.

▸ Click here for instructions to sign up for Postman services.

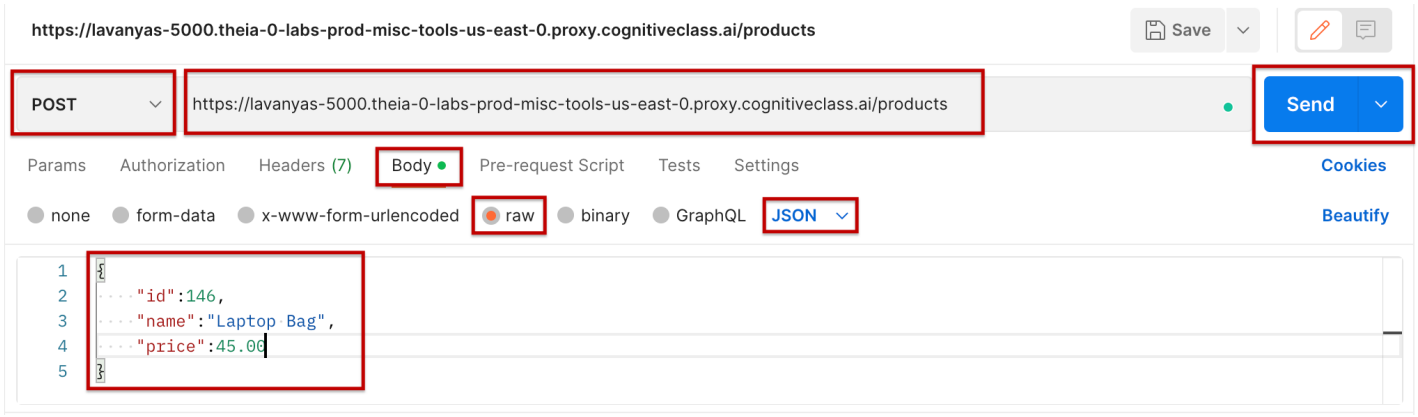
1. Click **Create New** to start creating a request to the REST API endpoint.



2. Choose **HTTP Request** from the options that you are presented with.



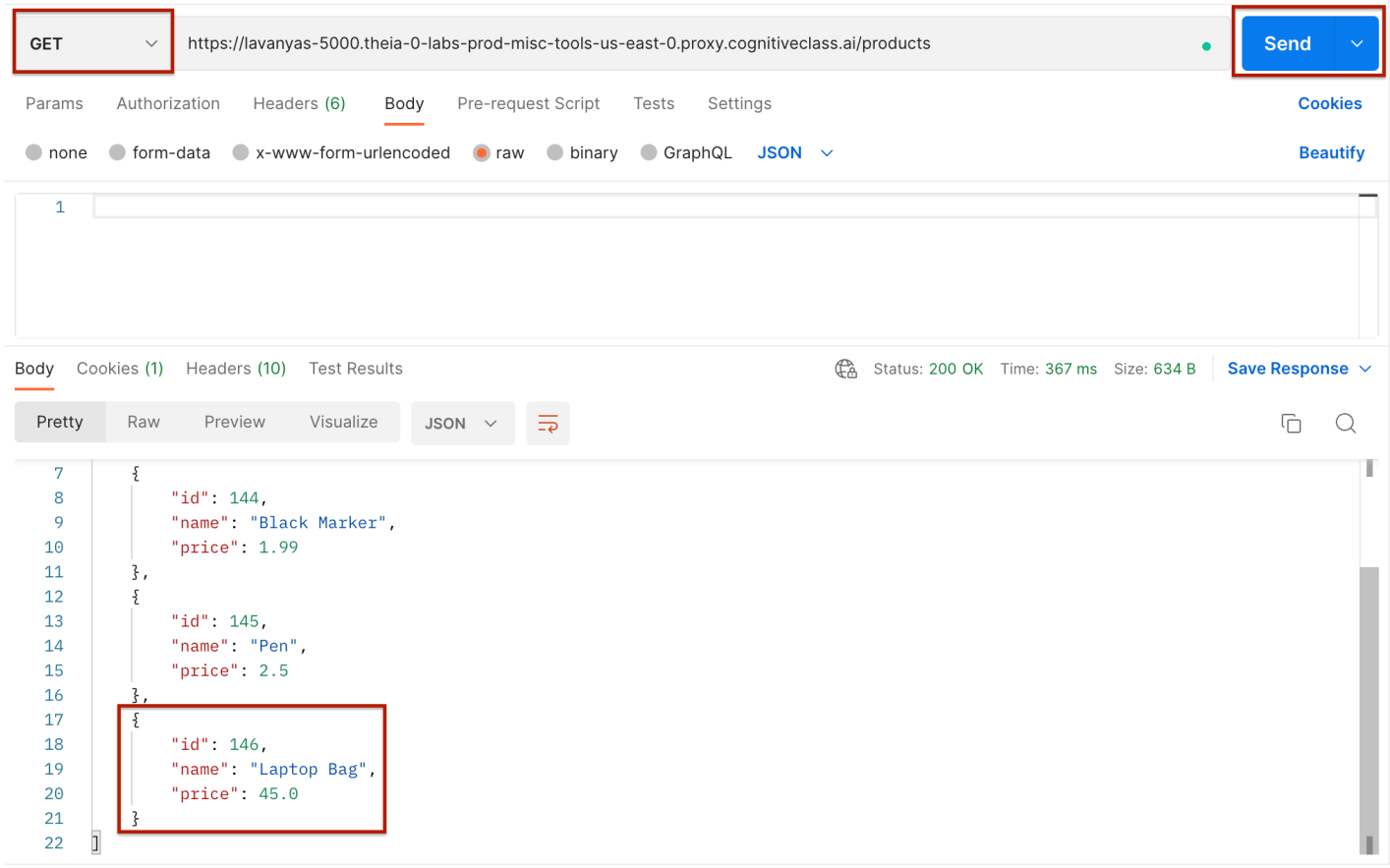
3. Paste the URL that you have copied into the address bar. Change the request type to `POST`. To send input as JSON, choose `raw->body->JSON`.



4. Enter the following JSON object and click the `Send` button. This will add the product to your previous list of products.

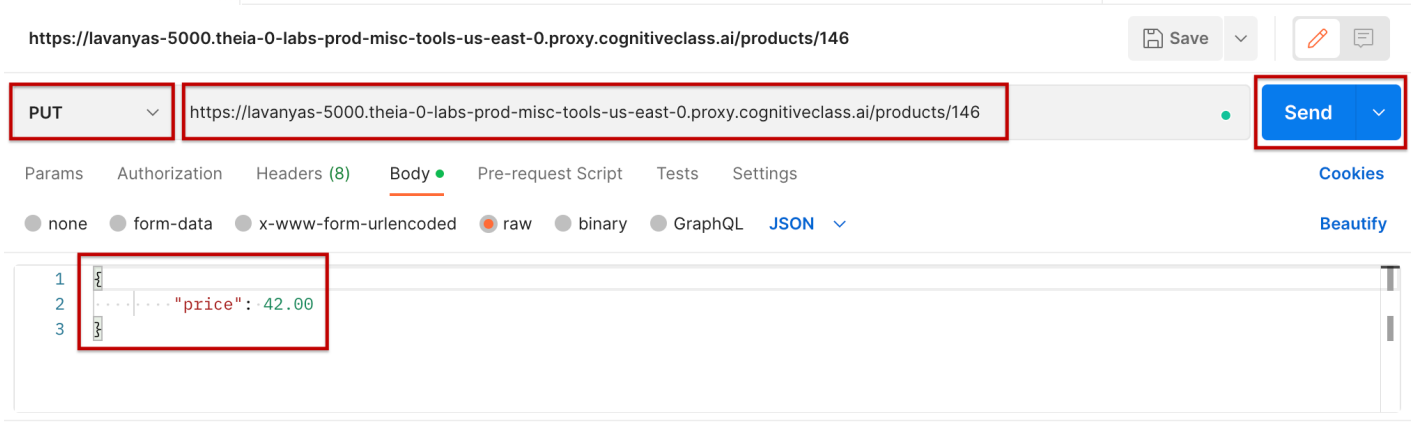
```
{
    "id":146,
    "name":"Laptop Bag",
    "price":45.00
}
```

5. Verify the list of products to ensure that the request has gone through and the product is added. Change the request type to `GET` and remove the JSON object from the request body. Click `Send` and observe the output in the response window.

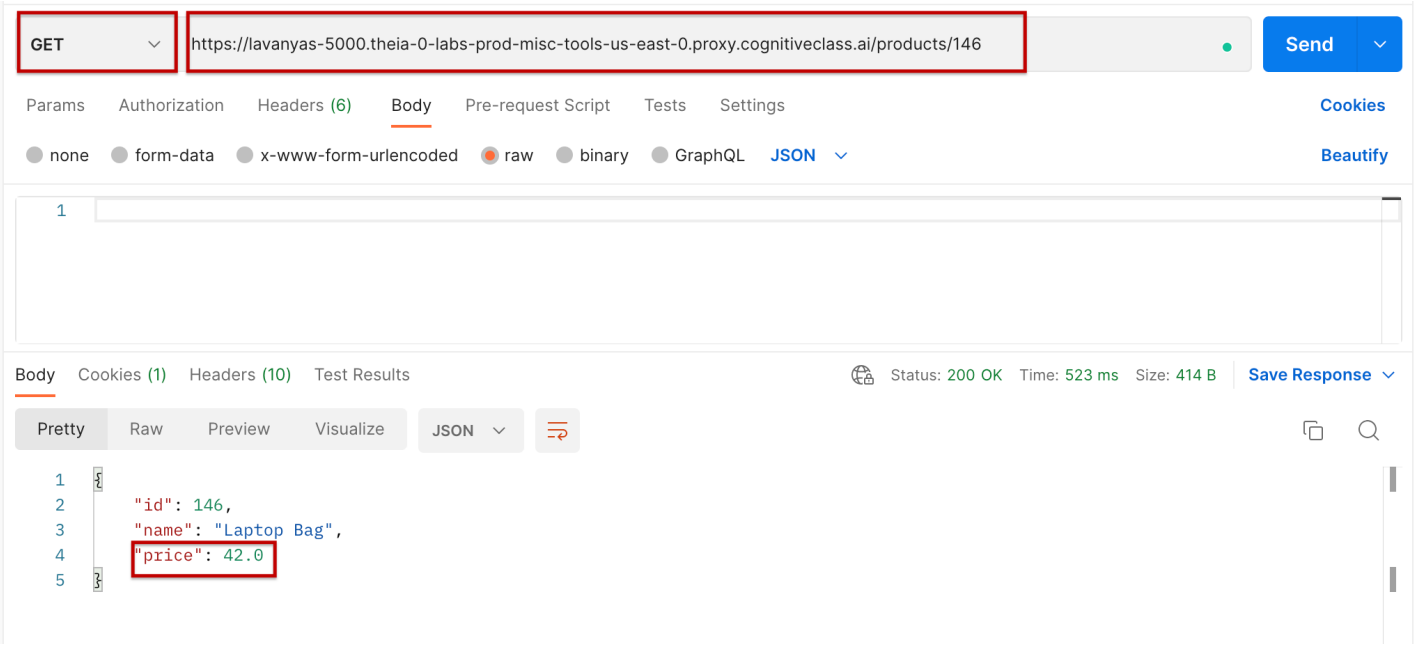


6. Test the PUT endpoint to update product details by changing the price of the item with id 146 to 42.00. Set the request type to PUT and add the product id to the end of the URL and add the value to be changed as a JSON body and click Send.

```
{
    "price":42.00
}
```



7. Verify the product with id 146 to ensure that the request has gone through and the product is updated. Change the request type to GET and remove the JSON object from the request body. Click Send and observe the output in the response window.



Practice labs

1. Update product details of product with id 144 by changing its price to 2.50.

▸ Click here for a hint
▸ Click here for the solution

2. Add the following product using POST method.

```
{
    "id":142,
    "name":"Eraser",
    "price":1.50
}
```

▸ Click here for a hint

3. Delete the product with id 142.

▸ Click here for a hint
▸ Click here for the solution

Congratulations! You have completed the lab for CRUD operations with Node.js and Express.js using CURL and Postman.

Summary:

In this lab, you have performed CRUD operations like GET, POST, PUT, and DELETE on a Node.js server application using Express.js and tested these methods using CURL and POSTMAN.

Author(s)

Lavanya T S